

SeqFeed: Improving LLM-Based RTL Code Generation with Sequential Behavior Feedback

Yuxin Du^{*1}, Juxin Niu^{*1}, Tao Hu¹,
Xi Wang^{2,3}, Zhe Jiang^{2,3}, Nan Guan¹

¹City University of Hong Kong,

²Southeast University, China,

³National Center of Technology Innovation for EDA, China

yuxindu8-c@my.cityu.edu.hk, juxin.niu@my.cityu.edu.hk, hu.tao@cityu.edu.hk,
xi.wang@seu.edu.cn, 101013615@seu.edu.cn, nanguan@cityu.edu.hk

Abstract

RTL code generation is a critical stage in hardware design, and the emergence of agentic systems offers new opportunities to automate this process. To generate correct RTL code, agents must understand sequential behavior, including how signals evolve and propagate over multiple clock cycles. However, effectively conveying such temporal information to agents remains a significant challenge. RTL source code does not directly reveal cycle-by-cycle signal evolution in a concrete execution, while full simulation waveforms are too large and contain too much irrelevant information for effective use by LLMs. To address these limitations, we study how human engineers reason about sequential behavior and identify three requirements for effective execution feedback: it should be event-addressable, dependency-traceable, and incrementally expandable. Guided by these requirements, we propose SeqFeed, a framework with two complementary components: (1) SeQuery, an SQL-like waveform query language that enables agents to anchor queries to semantic events and sample signal values at relative time points, and (2) SeGraph, a dependency graph that tracks signal propagation across clock cycles. Experimental results across multiple LLMs demonstrate the effectiveness of SeqFeed in improving pass rates. SeQuery and SeGraph are each effective independently and provide complementary benefits when used together. Further analysis indicates that SeqFeed shifts agent reasoning from speculation-driven conjecture to evidence-based analysis.

Introduction

The use of LLMs for complex tasks is shifting from static, single-turn prompting toward agentic problem solving, where an LLM agent autonomously invokes tools, reasons through multi-step trajectories, and iteratively refines its output through self-correction. This approach has been adopted across application domains including software engineering, scientific discovery, and task automation (Wang et al. 2024; Yao et al. 2023; Madaan et al. 2023). A central insight is that an agent’s ability to improve iteratively depends on whether its environment provides feedback that is both *informative*

and *actionable*. Such feedback should reveal how the current output deviates from the target and provide concrete guidance for correction (Gou et al. 2024).

RTL code generation is a critical stage in hardware design, translating functional specifications into synthesizable hardware description languages such as Verilog. Traditionally, this process has required substantial manual effort and remains susceptible to subtle logic errors (Abdollahi et al. 2025). Although LLMs have shown promising results in automating RTL code generation (Liu et al. 2023; Tsai, Liu, and Ren 2024; Chang et al. 2024), their integration into agentic workflows remains underexplored.

Producing correct RTL code requires an agent to understand its sequential behavior, including how signals evolve and propagate across clock cycles. To iteratively improve the generated RTL, the agent needs effective feedback that helps it understand such behavior and identify appropriate code revisions. However, providing this feedback in an effective way remains challenging. The two most direct sources of sequential information are RTL source code and simulation waveforms. These two sources provide complementary views of sequential behavior, but each has limitations when used for agents.

RTL source code describes the design logic and the rules governing state transitions. However, it does not directly reveal which path conditions hold or how signals evolve in a specific execution (Yao et al. 2025). A simulation mismatch indicates that the generated RTL does not satisfy the intended behavior, but it does not explain which sequential behavior caused the mismatch. As illustrated in Figure 1, without targeted cycle-level execution evidence, the agent may generate multiple competing hypotheses that are all plausible from the source code. It must then select one hypothesis, revise the RTL, and re-simulate the design. If the mismatch persists, the agent repeats the same process with another unsupported hypothesis, resulting in an inefficient cycle of conjecture, revision, and re-simulation.

Simulation waveforms complement RTL source code by providing concrete execution evidence, including cycle-level signal values and state transitions. Such evidence can help distinguish among hypotheses that cannot be resolved from

^{*}These authors contributed equally.

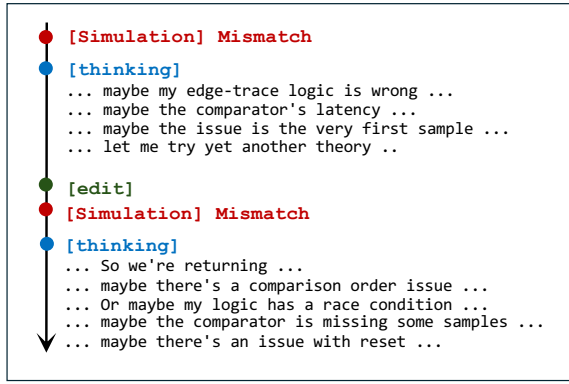


Figure 1: Without cycle-level evidence, an agent may repeatedly revise RTL based on unsupported hypotheses.

the source code and the simulation result alone. Recent work has demonstrated the potential of waveform feedback for LLM-based RTL generation (Ho, Ren, and Khailany 2025). However, full waveforms often contain many signals and cycles unrelated to the agent’s current reasoning, overwhelming the context window with unnecessary information. Besides, raw waveforms show what occurred during execution but do not directly connect the observed behavior to the responsible RTL logic or indicate how the code should be revised.

Another possible feedback mechanism is to instrument the RTL with diagnostic constructs such as `$display`. However, the instrumentation must be specified before simulation, and modifying the observed signals or trigger conditions requires rerunning the simulation. Because simulating a complex design can take minutes or even hours, repeated instrumentation and re-simulation can substantially slow the agent’s iterative refinement process.

Motivated by these challenges, we make the following three contributions:

- We derive three design requirements by examining how human engineers jointly use RTL source code and simulation waveforms to diagnose sequential behavior.
- We propose *SeqFeed*, which includes *SeQuery* for event-based waveform querying and *SeGraph* for cycle-aware dependency tracing.
- We evaluate *SeqFeed* on RTL code generation benchmarks across multiple LLMs, demonstrating that it improves success rates and enables more evidence-based diagnosis and refinement.

Motivation

Agentic RTL generation requires effective feedback about sequential behavior. To design such feedback, we draw on common practices used by hardware engineers to interpret sequential executions and distill three recurring patterns.

First, engineers anchor their inspection on events. Rather than scanning a waveform cycle by cycle, they typically begin by locating a specific event, such as the completion of a handshake, and then inspect its surrounding cycles. The

event provides a reference point, while the surrounding window provides context. *Second, engineers trace dependencies backward.* After identifying a signal whose behavior requires explanation, engineers trace the upstream assignments, enabling conditions, and earlier signal values that determine its value. They follow these dependencies backward across clock cycles, focusing the inspection on relevant signals at progressively earlier cycles. *Third, engineers refine their inspection scope iteratively.* Engineers seldom determine all relevant signals and cycles in advance. Instead, each observation informs a revised hypothesis, which suggests additional signals or cycles to inspect. The inspection scope thus expands progressively as their understanding develops.

Together, these patterns determine *where* to look, *what* is relevant, and *how much* information to examine. From them, we derive three requirements for a sequential feedback mechanism for LLM agents:

- *Event-addressable.* The mechanism should allow an agent to locate cycles in which a specified signal event occurs and inspect selected signals within a surrounding cycle window, without scanning the entire waveform.
- *Dependency-traceable.* Given a signal at a specific cycle, the mechanism should identify the RTL assignments, enabling conditions, and earlier signal values that determine its observed value.
- *Iteratively-queryable.* The mechanism should allow an agent to issue successive queries based on previously obtained evidence, without requiring the full inspection process to be specified in advance.

SeqFeed

Overview

SeqFeed provides effective sequential feedback to agents through *SeQuery* and *SeGraph*:

- *SeQuery* is a waveform query language. When an agent needs to answer questions such as “at which cycle did a handshake succeed, and what were the signal values around it,” it writes a query targeting the relevant event. Starting from a simple condition, the agent refines or expands the query as its debugging hypothesis evolves.
- *SeGraph* is a dependency graph extracted from RTL source code. Given a signal of interest, it returns an upstream subgraph organized by clock cycle: which assignments, conditions, and upstream signals contributed, and over how many cycles. The agent follows this dependency chain backward to the root cause.

SeqFeed does not prescribe a fixed invocation order between the two; the agent decides autonomously when to query the waveform, when to trace dependencies, and how to interleave them as hypotheses evolve. Figure 2 illustrates this process with a concrete example.

Consider a data-stream accumulator. The agent generates an initial RTL implementation and simulates it, finding a mismatch: input `[5, 10]` yields 20 instead of the expected 15. To locate the error, the agent first traces `out_data` backward through *SeGraph* and learns that

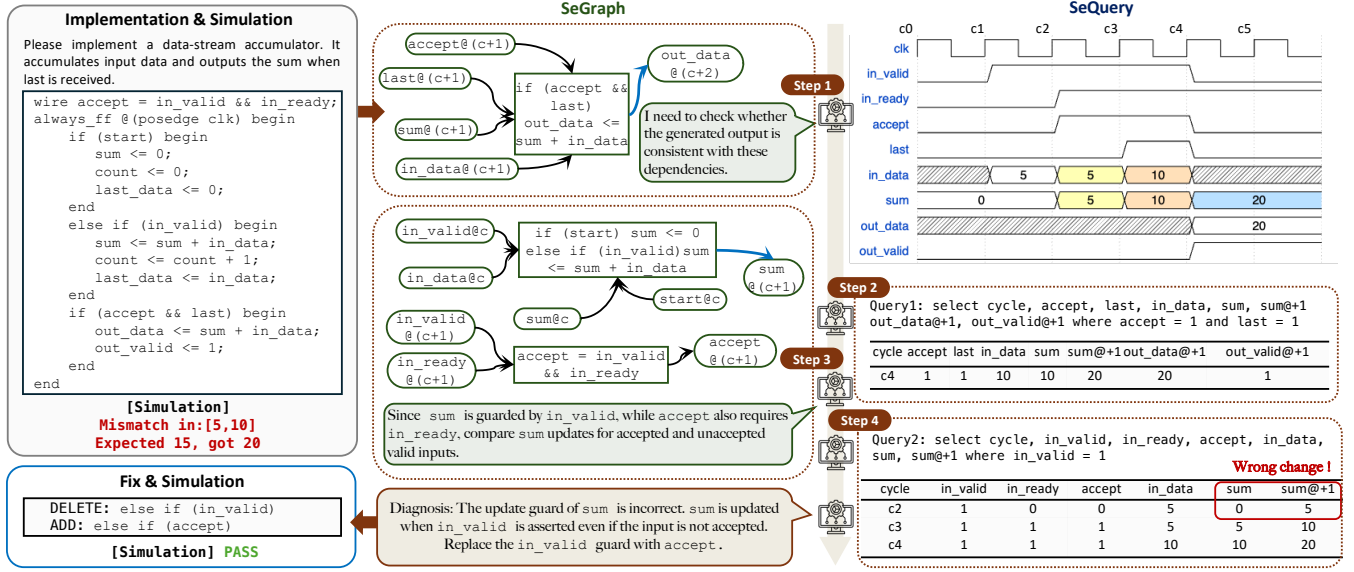


Figure 2: Overview of SeqFeed.

out_data = sum + in_data. It then issues a SeQuery at the cycle where the final input is accepted: sum = 10, in_data = 10, so out_data = 20 is consistent with that expression. Logically, however, sum should be 5 before processing the last input of 10, which means sum was already accumulated incorrectly earlier. Suspecting an error in how sum is updated, the agent traces sum’s dependencies through SeGraph and discovers that the gating condition is in_valid rather than accept. After confirming this finding with a second SeQuery, the agent corrects the gating condition to accept, regenerates the code, and the simulation passes.

The next two subsections detail SeQuery and SeGraph, respectively, while the final subsection discusses their design trade-offs between tool completeness and LLM usability.

SeQuery

To understand how a SeQuery maps to its result, consider Query 1 in Figure 2. The select clause specifies which signal values to retrieve for each matching cycle. The where clause restricts the result to cycles where accept = 1 and last = 1 hold simultaneously. The s@+1 suffix samples a signal one cycle after the matched cycle. The result is a table with one row per matched cycle and one column per select item.

Three core primitives emerge from this example. First, select defines the columns of the result table: for each matching cycle, the user specifies which signals and which offsets to observe. Second, where is a Boolean condition over signal values that filters anchor cycles; it supports comparisons, edge-detection functions, and and/or/not combinations. Third, the s@k notation samples signal s at an offset of k cycles relative to the anchor, with k > 0 looking forward and k < 0 looking backward. These primitives compose: more complex checks are assembled by adding conditions in where and offset samples in select.

Table 1 presents further examples of these primitives applied to data latching, backpressure, fixed-latency pipelines, FSM state transitions, and pipeline liveness.

Formally, a query is defined by the following grammar:

$Q ::= \text{select } I(, I)^* \text{ where } P$

$I ::= \text{cycle} \mid s@k \mid \text{min}(\text{cycle}) \mid \text{max}(\text{cycle}) \mid \text{count}(*)$

$P ::= E \bowtie E \mid \text{rise}(s) \mid \text{fall}(s) \mid \text{change}(s) \mid \text{not } P \mid (P) \mid P \wedge P \mid P \vee P$

$E ::= \text{cycle} \mid s@k \mid c$

In the grammar above, s denotes a signal name, $k \in \mathbb{Z}$ is an integer cycle offset, c is a constant, and $\bowtie \in \{=, \neq, <, \leq, >, \geq\}$ is a comparison operator. Evaluation proceeds under an externally specified clock domain and active edge. The evaluator enumerates all clock cycles within the query range, treating each cycle in turn as the anchor for where predicate evaluation. Signal values are sampled before the active clock edge; s@k denotes the sampled value of signal s at k cycles relative to the current anchor. For brevity, a bare signal name defaults to s@0. Comparisons operate on four-state logic values (0, 1, X, Z). Among the edge-detection functions, change(s) tests whether adjacent samples differ in any way, while rise(s) and fall(s) detect only strict single-bit 0 → 1 and 1 → 0 transitions. Aggregate functions such as min(cycle) and count(*) let the agent summarize the set of matching cycles, for instance, identifying the first or last occurrence of an event.

SeGraph

To understand what SeGraph provides, consider the out_data dependency chain in Figure 2. Signals accept and last feed into the condition node accept && last, which gates the computation node sum + in_data; the result is assigned to out_data. This chain is organized

Table 1: SeQuery examples across five sequential checking scenarios.

Natural language description	Query
<i>Detect data latch errors.</i> When a transfer is accepted at cycle t , the latched data at $t + 1$ must match the input at t .	select cycle, datain, data_latched@ + 1 where valid = 1 $\wedge$ ready = 1 $\wedge$ data_latched@ + 1 $\neq$ datain
<i>Check data stability under backpressure.</i> When valid = 1 and ready = 0, the source must hold both valid and data stable until the transfer completes.	select cycle, valid, ready, valid@ + 1, data, data@ + 1 where valid = 1 $\wedge$ ready = 0 $\wedge$ (valid@ + 1 $\neq$ 1 $\vee$ data@ + 1 $\neq$ data)
<i>Check a fixed-latency pipeline.</i> An input accepted at t should produce valid output at $t + 3$.	select cycle, in_data, out_valid@ + 3, out_data@ + 3 where in_valid = 1 $\wedge$ in_ready = 1 $\wedge$ (out_valid@ + 3 $\neq$ 1 $\vee$ out_data@ + 3 $\neq$ in_data)
<i>Check FSM state transitions.</i> A transition from state 0 to 1 is allowed only when start = 1.	select cycle, state, state@ + 1, start where state = 0 $\wedge$ state@ + 1 = 1 $\wedge$ start $\neq$ 1
<i>Detect pipeline bubble stalls.</i> A pipeline expected to produce every cycle must not idle for 3 or more consecutive cycles.	select cycle, out_valid, out_valid@ + 1, out_valid@ + 2 where out_valid = 0 $\wedge$ out_valid@ + 1 = 0 $\wedge$ out_valid@ + 2 = 0

by clock cycle rather than by RTL source line order. The assignment of `accept`, the test of `last`, and the computation of `sum` appear in separate code locations, yet every node carries a cycle annotation that makes cross-cycle value propagation visually explicit. This cycle-based organization contrasts with conventional dependency graphs and AST-based representations, which follow code structure.

Three structural properties emerge from this example.

- *Node.* SeGraph uses two logical node types internally. A *signal node* represents an RTL signal. A *statement node* captures the RTL code that produces one or more signals, including conditions, computations, and control structures (`if/case`). When presented to the agent, each statement node is expanded: conditions become condition nodes (diamonds in Figure 2), computations become computation nodes (rectangles), and both connect to their signal operands (rounded rectangles). This expansion yields the three visual forms shown in the figure.
- *Edge.* All dependencies form alternating chains of signal and statement nodes. A statement node reads its input signals, evaluates conditions, performs computations, and assigns the results to output signals. Both value operands (e.g., `sum` and `in_data` feeding the addition in the `out_data` chain) and control conditions (e.g., `accept` and `last` forming the gating condition) are treated as statement inputs; the complete causal context is thus preserved within the graph.
- *Cycle-based organization.* SeGraph groups nodes by the clock cycle to which they belong. A combinational statement and its input and output signals all reside within a single cycle group, as in the `out_data` chain of Figure 2. A clocked statement marks a cycle transition: its input signals belong to cycle n and its output signals to cycle $n + 1$. In the `sum` update subgraph of Figure 2, `sum` and `in_data` combine in the current cycle and the result becomes `sum@+1` in the next, crossing the cycle boundary. A multi-cycle dependency therefore appears as a path through successive cycle groups, making data propagation across clock ticks directly traceable.

Algorithm 1 Construction of SeGraph.

Require: RTL design D

Ensure: $G = (V_{\text{sig}} \cup V_{\text{stmt}}, E)$

- 1: Parse continuous assignments, procedural assignments, and control structures in D
- 2: $V_{\text{sig}} \leftarrow \{x \mid x \text{ is a signal in } D\}$
- 3: $V_{\text{stmt}} \leftarrow \emptyset, E \leftarrow \emptyset$
- 4: **for** each combinational or clocked process P in D **do**
- 5: **for** each signal y updated by P **do**
- 6: Create a statement node b from y 's update logic and enclosing `if/case` conditions
- 7: Mark b as combinational or clocked
- 8: $V_{\text{stmt}} \leftarrow V_{\text{stmt}} \cup \{b\}$
- 9: $X \leftarrow \{x \in V_{\text{sig}} \mid x \text{ is read by } b\}$
- 10: $E \leftarrow E \cup \{(x, b) \mid x \in X\} \cup \{(b, y)\}$
- 11: **return** G

These structural properties are formalized in a static graph $G = (V, E)$, built once from the RTL design. V partitions into signal nodes V_{sig} and statement nodes V_{stmt} ; edges alternate between the two sets, so $E \subseteq (V_{\text{sig}} \times V_{\text{stmt}}) \cup (V_{\text{stmt}} \times V_{\text{sig}})$. Given a target signal $s \in V_{\text{sig}}$, SeGraph extracts a subgraph by backward traversal: from a signal to its producing statements, from a statement to its input signals. Traversal terminates at primary inputs, undefined signals, visited nodes, or a depth limit. Symbolic unfolding is applied during traversal: combinational statements preserve the current cycle, while clocked statements advance it by one. The result is a cycle-organized chain, as shown in Figure 2. Algorithm 1 constructs G from a parsed RTL design D by mapping signals to V_{sig} , update logic to V_{stmt} (marked combinational or clocked), and read/write dependencies to E .

Discussion

The design of SeQuery and SeGraph reflects a tradeoff between tool completeness and LLM usability. Formal completeness requires covering every temporal expression, but

Table 2: Distribution of benchmark cases across categories.

Category	#	Category	#
Digital Signal Processing	55	Memory & Bus	30
Machine Learning	57	Control-Driven	32
Accelerator	58	Heavy IP Integration	24

the resulting syntax and semantics are difficult for LLMs to use reliably. Conversely, optimizing purely for ease of use risks leaving important verification scenarios uncovered. SVA illustrates the former problem: although it can express arbitrary temporal properties, LLMs are prone to producing erroneous SVA assertions (Pulavarthi et al. 2025; Pulavarthi, Nandal, and Dan 2025), which limits its practical value in agentic workflows. Our design navigates this tension through the following choices:

- SeQuery builds on SQL syntax and introduces only one additional construct beyond standard select-where: the $s@k$ relative-cycle offset. This keeps the learning cost near zero for LLMs while enabling cross-cycle queries. As Table 1 shows, this minimal design already covers a range of common sequential verification needs.
- SeGraph unifies signals, assignment statements, and conditional branches into a single dependency graph, capturing the dominant causal structures in RTL. An LLM agent specifies only the signals of interest and receives a sub-graph organized by clock cycle, which makes cross-cycle value propagation visually interpretable without reasoning over raw source code.

This design does leave some scenarios uncovered, such as multi-clock-domain designs. Overall, it represents a deliberate choice to favor LLM accessibility at the cost of some completeness, with the aim of enabling agents to obtain stable, actionable sequential feedback during iterative debugging.

Evaluation

Experimental Setup

Our benchmark combines cases from existing RTL generation benchmarks, including CVDP (Pinckney et al. 2025) and RTLLM (Lu et al. 2024), with hardware design cases from recent open-source work (Hu et al. 2025; Wanna, Chen, and Hao 2025). We exclude purely combinational circuits and toy cases that do not arise in real design practice, keeping the benchmark representative of practical hardware design. Table 2 shows the distribution of the 256 cases across six categories.

We evaluate on six model settings: DeepSeek Flash (no-think and think modes), DeepSeek Pro (no-think and think modes), and MiniMax M3 (no-think and think modes). For each model and configuration, we run all 256 benchmark cases. To isolate the contribution of each component, we compare four configurations: (1) baseline, where the agent receives only simulation error messages and neither SeQuery nor SeGraph are available; (2) SeQuery only; (3) SeGraph only; (4) SeqFeed, where both components are available. All other settings remain identical. Agentic RTL generation is a

long-horizon task spanning multiple rounds of iteration, each consuming tokens for context and generation. We compare configurations by token cost: under a fixed token budget per case, we record whether the agent produces a passing design.

We use OpenCode (Anomaly 2025) as the agent framework, with up to 65 steps per case. Simulation uses Icarus Verilog, and test frameworks are built with cocotb (cocotb contributors 2025). Each agent run executes in an isolated Docker container with access only to the design specification and the test framework. A case passes if the final RTL produced within the step budget passes all cocotb tests.

Main Results

Figure 3 reports the pass rate for each model setting under the four configurations. SeqFeed improves pass rate over Baseline across all six model settings. The gain ranges from 8.7 percentage points on DeepSeek Pro think (from 60.4% to 69.1%) to 18.3 percentage points on DeepSeek Flash think (from 53.0% to 71.3%). In absolute terms, SeqFeed achieves the highest pass rate on DeepSeek Flash think (71.3%), followed by MiniMax M3 think (70.9%) and DeepSeek Pro think (69.1%). The improvement persists across both Flash and Pro tiers and across both think and no-think modes. Structured sequential feedback provides gains independent of model capacity and reasoning mode: even the strongest baseline, DeepSeek Pro think at 60.4%, benefits from SeqFeed. Ablation study shows that both SeQuery and SeGraph independently contribute to the pass rate gain. With SeQuery alone, pass rate rises above Baseline by 2.2 to 6.1 percentage points across the six model settings. With SeGraph alone, the gain ranges from 2.1 to 9.2 percentage points. The full SeqFeed configuration achieves the highest pass rate on every model setting, reaching 71.3% on DeepSeek Flash think. On every model setting, the SeqFeed gain over Baseline (8.7–18.3 percentage points) exceeds the sum of the SeQuery-only and SeGraph-only gains, indicating that the two components reinforce each other.

Token Utilization

Figure 4 plots cumulative pass rate against cumulative token cost. A flat segment indicates that further token spending produces no new passes. Across all six settings, Baseline’s gains are concentrated in the first 100K tokens and then nearly stop. DeepSeek Flash Think is representative: Baseline gains 46.5 percentage points in the first 100K, then only 6.9 points across the remaining 679K. SeqFeed gains 57.8 points in the first 100K and continues at 13.5 points across the subsequent range, roughly double Baseline’s post-100K pace. The flat region of each Baseline curve measures debugging iterations that consume tokens without converging.

The MiniMax M3 No-Think curves make the trade-off explicit. Baseline leads early (13.5% at 10K tokens against SeqFeed’s 6.1%) but stops gaining after 30K, finishing at 41.7%. SeqFeed overtakes and reaches 57.0%. This is the only setting where Baseline starts ahead; it is also where Baseline’s productive range is narrowest. Structured feedback imposes a measurable upfront cost, but that cost buys access to cases that simple retry strategies cannot solve.

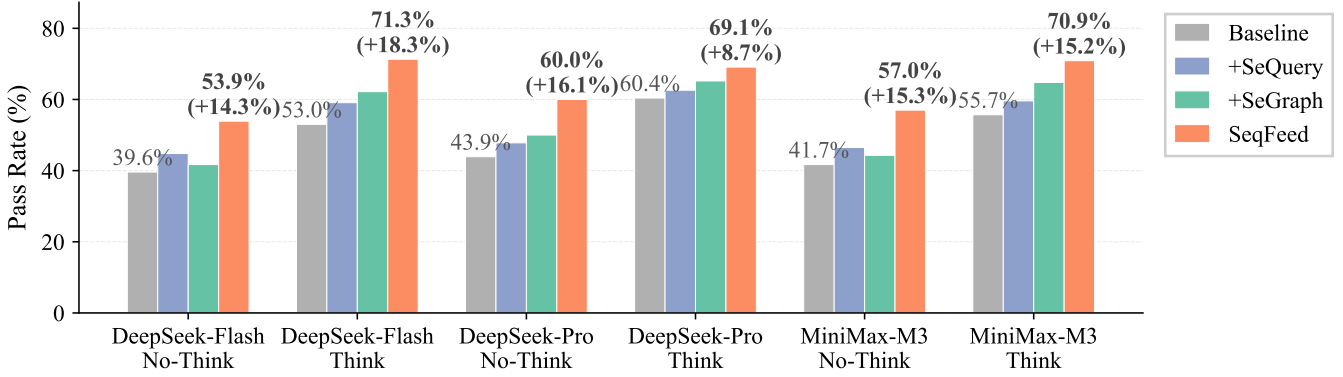


Figure 3: Pass rate across model configurations.

Tool Use Density

To characterize how the agent allocates its token budget between reasoning and action, we define *tool use density* as the number of tool invocations per output token:

$$\text{tool use density} = \frac{\# \text{tool invocations}}{\# \text{output tokens}}.$$

We count only tool calls that advance the iterative RTL generation loop: editing the design, running simulation, and querying SeQuery or SeGraph. Calls unrelated to iteration (e.g., reading the specification or listing files) are excluded, as they do not reflect the debugging strategy. Figure 5 reports tool use density across the six model settings, normalized so that each model’s Baseline equals 1.0. A value above 1.0 means the configuration calls tools at a higher rate per token than Baseline; a value below 1.0 means it calls them at a lower rate. The results reveal a sharp split by reasoning mode:

- *No-Think models.* On these three settings, SeqFeed raises tool use density substantially, to $2.4\text{--}3.9\times$ the Baseline rate. Without a thinking mode, these models debug primarily through text: they generate hypotheses in natural language, edit the design, and simulate. Tool calls are sparse because the available tools do not provide waveform feedback; the agent fills the gap with internal reasoning. SeqFeed provides concrete entry points for action: SeQuery tests a hypothesis about a specific cycle, and SeGraph traces the dependency chain. The agent responds by invoking tools more frequently per token, replacing speculative reasoning with evidence-seeking queries.
- *Think models.* On all three think settings, SeqFeed *lowers* tool use density to $0.62\text{--}0.84\times$ Baseline. With thinking mode, these models already reason about their debugging strategy in structured thought tokens. In Baseline, they make tool calls, but each call yields limited information: without waveform access, the agent probes repeatedly to narrow down a bug. SeqFeed makes each call more informative. A single SeQuery can confirm or refute a hypothesis that would otherwise require several speculative edit and simulate cycles. The per-token tool call rate drops because each call yields more information per query. DeepSeek Pro Think illustrates the shift: pass rate

rises from 60.4% to 69.1% while tool calls per token fall to $0.62\times$ Baseline.

Related Work

LLM-Aided Hardware Design Hardware design translates functional specifications into RTL implementations and is central to modern chip development. LLMs have been applied across the EDA pipeline, including RTL generation, verification, and optimization (Zhong et al. 2023; Pan et al. 2025). The agentic paradigm has been introduced to RTL tasks, where LLM agents autonomously invoke tools and iteratively refine their output (Fang et al. 2026; Chang et al. 2026). For RTL generation, prior work uses iterative compile-and-simulate workflows (Thakur et al. 2024b; Ranga et al. 2024; Zhao et al. 2025), fine-tunes LLMs on Verilog datasets (Thakur et al. 2024a; Chang et al. 2024), and introduces benchmarks for evaluating generated RTL (Lu et al. 2024; Pinckney et al. 2025). Other efforts target Chisel generation (Zhao et al. 2026; Niu et al. 2025) or transpile C/Python into synthesizable Verilog (Liao, Adegbiya, and Lysecky 2025; Firouzi et al. 2025).

Sequential Feedback Effective feedback is essential for LLM agents to iteratively refine their outputs (Gou et al. 2024; Madaan et al. 2023). Software engineering agents commonly rely on compiler diagnostics and unit-test outcomes to identify program-level errors (Chen et al. 2024; Shinn et al. 2023). RTL agents similarly use compiler errors, simulation results, EDA reports, and waveform traces (Tsai, Liu, and Ren 2024; Xu et al. 2025; Ho, Ren, and Khailany 2025; Blocklove et al. 2025). In particular, systems such as VerilogCoder (Ho, Ren, and Khailany 2025) and BlockLove (Blocklove et al. 2025) show that waveform slices and timing diagrams can improve RTL correction by exposing cycle-level behavior. However, existing waveform feedback is typically preselected and static, preventing agents from querying transitions of interest, adjusting the temporal scope, or following signal dependencies across cycles. This limitation is compounded by evidence that LLMs continue to struggle with state transitions and cross-cycle dependencies (Pulavarthi et al. 2025; Pulavarthi, Nandal, and Dan 2025). Providing sequential feedback that agents can query and explore according to their

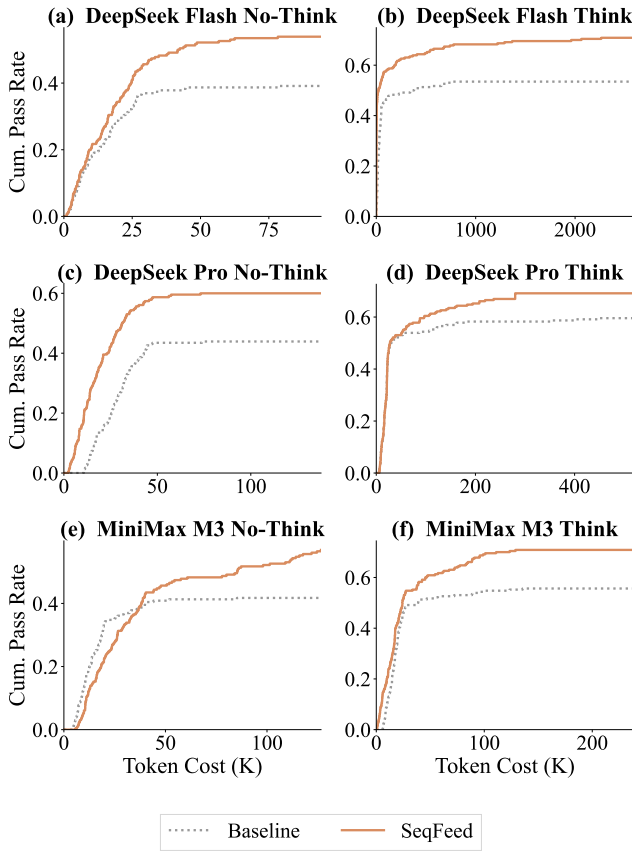


Figure 4: Cumulative pass rate against cumulative token cost.

evolving hypotheses therefore remains an open challenge.

Conclusion

We identified the absence of effective sequential feedback as a central bottleneck in agentic RTL generation. By examining how human engineers interpret sequential behavior, we derived three requirements for such feedback: event-addressability, dependency-traceability, and incremental expandability. To meet these requirements, we built SeqFeed, which combines SeQuery, a waveform query language, with SeGraph, a cycle-organized dependency graph. Experiments across six LLM configurations show that SeqFeed improves pass rate and token efficiency, with the two components independently effective and complementary. Analysis of agent reasoning further indicates a shift from speculation-driven conjecture to evidence-driven diagnosis.

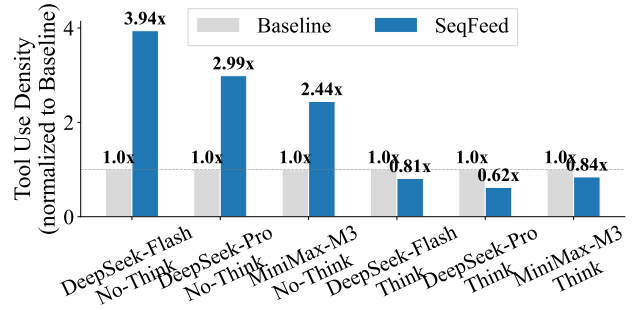


Figure 5: Tool use density across model configurations.

References

- Abdollahi, M.; Yeganli, S. F.; Baharloo, M. A.; and Bani-asadi, A. 2025. Hardware Design and Verification with Large Language Models: A Scoping Review, Challenges, and Open Issues. *Electronics*, 14(1).
- Anomaly. 2025. OpenCode: The Open Source AI Coding Agent. <https://github.com/anomalyco/opencode>. Accessed: 2026-07-27.
- Blocklove, J.; Thakur, S.; Tan, B.; Pearce, H.; Garg, S.; and Karri, R. 2025. Automatically Improving LLM-based Verilog Generation using EDA Tool Feedback. *ACM Trans. Des. Autom. Electron. Syst.*, 30(6).
- Chang, F.-C.; Yang, Y.-H.; Huang, H.-M.; Hsu, Y.-C.; Lin, Y.-Y.; Tsai, M.-F.; Yang, C.-C.; and Wu, P.-Y. 2026. SpecLoop: An Agentic RTL-to-Specification Framework with Formal Verification Feedback Loop. arXiv:2603.02895.
- Chang, K.; Wang, K.; Yang, N.; Wang, Y.; Jin, D.; Zhu, W.; Chen, Z.; Li, C.; Yan, H.; Zhou, Y.; Zhao, Z.; Cheng, Y.; Pan, Y.; Liu, Y.; Wang, M.; Liang, S.; Han, Y.; Li, H.; and Li, X. 2024. Data is all you need: Finetuning LLMs for Chip Design via an Automated design-data augmentation framework. In *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC '24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400706011.
- Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*.
- cocotb contributors. 2025. cocotb: Python-based Chip RTL Verification. <https://github.com/cocotb/cocotb>. Accessed: 2026-07-27.
- Fang, W.; Lu, Y.; Liu, S.; Wang, J.; Guo, Z.; He, J.; Tu, F.; and Xie, Z. 2026. Dr. RTL: Autonomous Agentic RTL Optimization through Tool-Grounded Self-Improvement. arXiv:2604.14989.
- Firouzi, F.; Nakkilla, S. S. R.; Fu, C.; Banerjee, S.; Talukdar, J.; and Chakrabarty, K. 2025. LLM-AID: Leveraging Large Language Models for Rapid Domain-Specific Accelerator Development. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, IC-CAD '24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400710773.

- Gou, Z.; Shao, Z.; Gong, Y.; yelong shen; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*.
- Ho, C.-T.; Ren, H.; and Khailany, B. 2025. VerilogCoder: autonomous verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI'25/IAAI'25/EAAI'25*. AAAI Press. ISBN 978-1-57735-897-8.
- Hu, Y.; Ye, J.; Xu, K.; Sun, J.; Zhang, S.; Jiao, X.; Pan, D.; Zhou, J.; Wang, N.; Shan, W.; Fang, X.; Wang, X.; Guan, N.; and Jiang, Z. 2025. UVLLM: An Automated Universal RTL Verification Framework using LLMs. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 1–7.
- Liao, Y.; Adegbiya, T.; and Lysecky, R. 2025. Are LLMs Any Good for High-Level Synthesis? In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD '24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400710773.
- Liu, M.; Pinckney, N.; Khailany, B.; and Ren, H. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 1–8.
- Lu, Y.; Liu, S.; Zhang, Q.; and Xie, Z. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 722–727.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. SELF-REFINE: iterative refinement with self-feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*. Red Hook, NY, USA: Curran Associates Inc.
- Niu, J.; Liu, X.; Niu, D.; Wang, X.; Jiang, Z.; and Guan, N. 2025. ReChisel: Effective Automatic Chisel Code Generation by LLM with Reflection. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 1–7.
- Pan, J.; Zhou, G.; Chang, C.-C.; Jacobson, I.; Hu, J.; and Chen, Y. 2025. A Survey of Research in Large Language Models for Electronic Design Automation. *ACM Trans. Des. Autom. Electron. Syst.*, 30(3).
- Pinckney, N.; Deng, C.; Ho, C.-T.; Tsai, Y.-D.; Liu, M.; Zhou, W.; Khailany, B.; and Ren, H. 2025. Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification. arXiv:2506.14074.
- Pulavarthi, V.; Nandal, D.; and Dan, D., Soham and Pal. 2025. AssertionBench: A Benchmark to Evaluate Large-Language Models for Assertion Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 8073–8080. Albuquerque, New Mexico: Association for Computational Linguistics. ISBN 979-8-89176-195-7.
- Pulavarthi, V.; Nandal, D.; Dan, S.; and Pal, D. 2025. Are LLMs Ready for Practical Adoption for Assertion Generation? In *2025 Design, Automation & Test in Europe Conference (DATE)*, 1–7.
- Ranga, S.; Mao, R.; Bhattacharjee, D.; Cambria, E.; and Chattopadhyay, A. 2024. RTL Agent: An Agent-Based Approach for Functionally Correct HDL Generation via LLMs. In *2024 IEEE 33rd Asian Test Symposium (ATS)*, 1–6.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*. Red Hook, NY, USA: Curran Associates Inc.
- Thakur, S.; Ahmad, B.; Pearce, H.; Tan, B.; Dolan-Gavitt, B.; Karri, R.; and Garg, S. 2024a. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Trans. Des. Autom. Electron. Syst.*, 29(3).
- Thakur, S.; Blocklove, J.; Pearce, H.; Tan, B.; Garg, S.; and Karri, R. 2024b. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887.
- Tsai, Y.; Liu, M.; and Ren, H. 2024. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC '24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400706011.
- Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; Zhao, W. X.; Wei, Z.; and Wen, J.-R. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*, 18(6).
- Wanna, A.; Chen, H.; and Hao, C. 2025. ForgeBench: A Machine Learning Benchmark Suite and Auto-Generation Framework for Next-Generation HLS Tools. arXiv:2504.15185.
- Xu, K.; Sun, J.; Hu, Y.; Fang, X.; Shan, W.; Wang, X.; and Jiang, Z. 2025. MEIC: Re-thinking RTL Debug Automation using LLMs. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD '24*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400710773.
- Yao, B.; Wang, N.; Liu, X.; Du, Y.; Hu, Y.; Gao, H.; Jiang, Z.; and Guan, N. 2025. ARSP: Automated Repair of Verilog Designs via Semantic Partitioning. arXiv:2508.16517.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
- Zhao, Y.; Huang, D.; Li, C.; Jin, P.; Song, M.; Xu, Y.; Nan, Z.; Gao, M.; Ma, T.; Qi, L.; Pan, Y.; Zhang, Z.; Zhang, R.; Zhang, X.; Du, Z.; Guo, Q.; and Hu, X. 2026. CodeV: Empowering LLMs With HDL Generation Through Multilevel Summarization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 45(4): 1893–1906.

Zhao, Y.; Zhang, H.; Huang, H.; Yu, Z.; and Zhao, J. 2025. MAGE: A Multi-Agent Engine for Automated RTL Code Generation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 1–7.

Zhong, R.; Du, X.; Kai, S.; Tang, Z.; Xu, S.; Zhen, H.-L.; Hao, J.; Xu, Q.; Yuan, M.; and Yan, J. 2023. LLM4EDA: Emerging Progress in Large Language Models for Electronic Design Automation. *arXiv:2401.12224*.